BUAP

Tarea: QDN

Aprendizaje por Refuerzo

RL para trading

Jorge Uc • 13 de octubre 2025

Estructura del proyecto

Para correr el modelo se utilizó el repositorio de proyectos de Github, en el repositorio se tiene la siguiente estructura de archivos:

- Chapter10/
  - data/
    - ch10-small-quotes.tgz
    - unpack_data.sh
  - lib/
    - __init__.py
    - common.py
    - data.py
    - environ.py
    - models.py
    - validation.py
  - test/
    - test_data.py
    - test_environ.py
  - conftest.py
  - run_model.py
  - train_model.py
  - train_model_conv.py

Lo primero que se realizó fue descomprimir el archivo h10-small-quotes.tgz, una vez realizado, se obtiene dos archivos, el YNDX_150101_151231.csv el cual contiene los datos del mercado de la compañia Yandex del 2015 y el YNDX_160101_161231.csv con los datos del 2016 de la misma compañia.

MODELO BASE (10 OBSERVACIONES)

Para entrenar el modelo, se ejecuta el train_model.py, el cual admite los siguientes parámetros: `--data` (archivo de entrenamiento), `--val` (archivo de validación) y `-r` (nombre de la ejecución, **obligatorio**). Por defecto, el script utiliza los datos de 2016 para el entrenamiento y los de 2015 para la validación, aunque estos pueden modificarse mediante los parámetros mencionados.
Se ejecuta el código con el comando:

```
python train_model.py -r simple-modelo_10bar
```

Es importante notar que el **el código nunca se detendrá**, por lo que hay que dejarlo un tiempo corriendo, en nuestro caso ambos modelos dejaron un total de 12 horas.
Una vez finalizado el entrenamiento se crean automáticamente dos carpetas runs y saves, en esta última se encuentra nuestro modelo.

Para evaluarlo necesitamos correr el archivo run_model.py el cual tiene los siguientes parámetros: `-d` (datos con los cuales se probará el modelo) `-m` (modelo que se usará) `-b` (números de observaciones) `-n` (nombre del archivo) y `--commission` (comisión que se usará, por defecto 0.1).
Probándolo con los datos de entrenamiento el comando es el siguiente:

```
python run_model.py -d data/YNDX_160101_161231.csv -m
saves/simple-modelo_10bar/mean_value-0.367.data -b 10 -n YNDX16
```

Para los datos de validación el comando es el siguiente:

```
python run_model.py -d data/YNDX_150101_151231 -m
saves/simple-modelo_10bar/mean_value-0.367.data -b 10 -n YNDX15
```

Estos códigos nos generan imágenes con los nombres que le dimos, en los cuales podemos ver el procentaje de ganancias que produce el modelo.

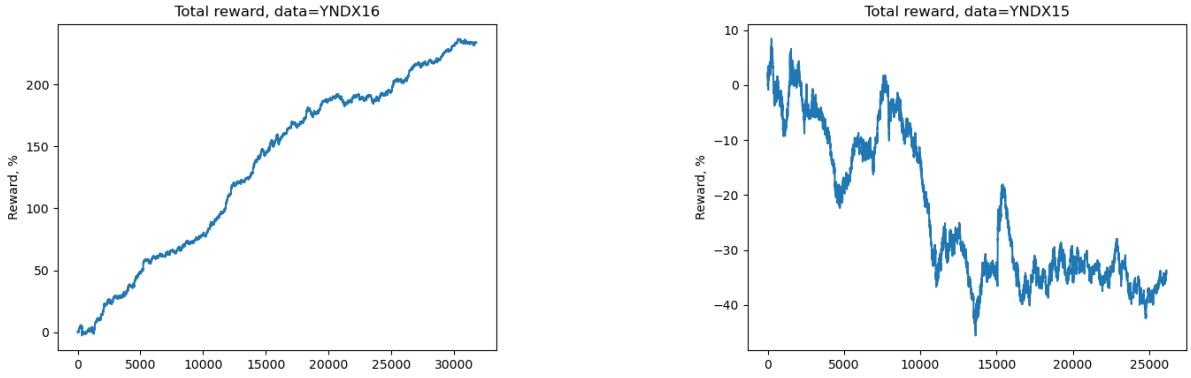


Figura 1: Beneficio comercial del modelo con 10 observaciones en los datos de entrenamiento (izquierda) y datos de validación (derecha)

Si bien es cierto que el modelo tiene no tuvo un desarrollo adecuado para los datos de validación, podemos apreciar que esto se debe a la comisión puesto que si variamos la comisión con el parámetro `--commission` se obtienen los siguientes valores:

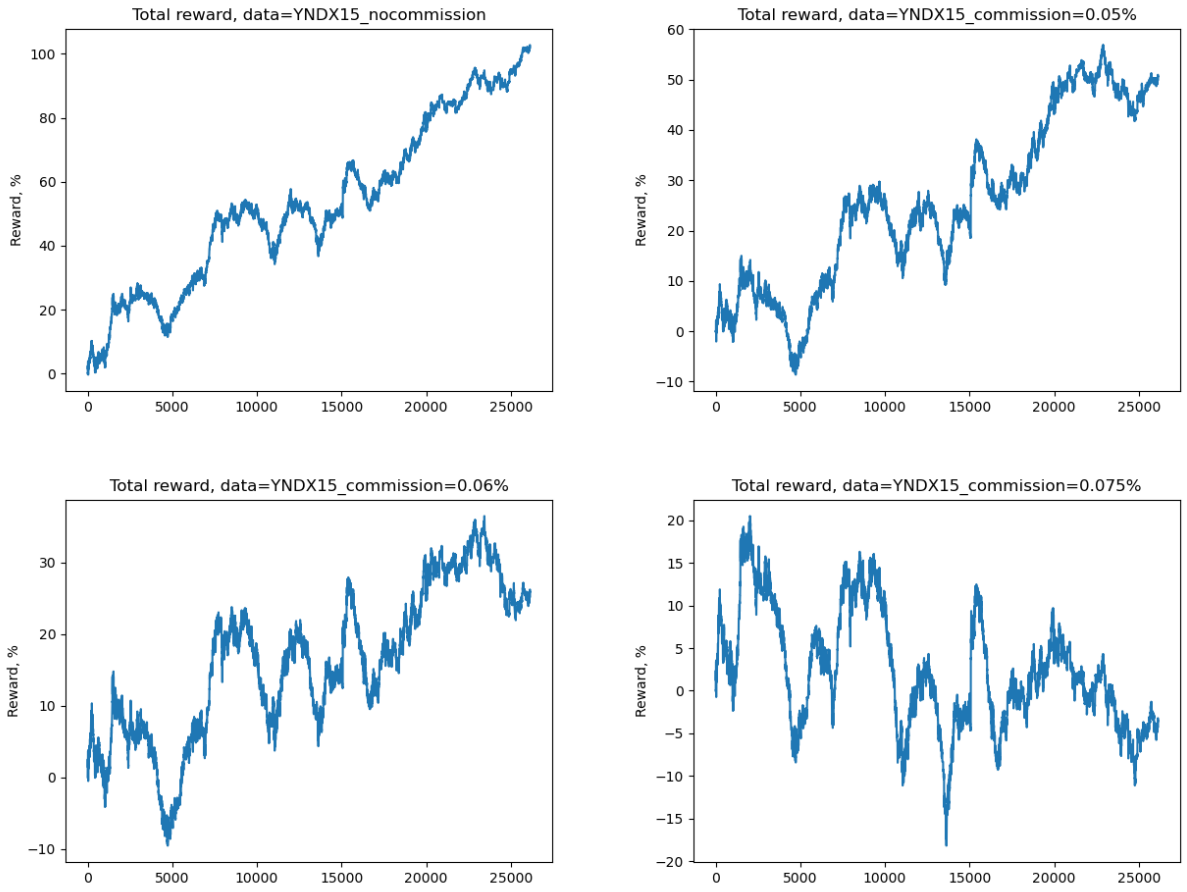


Figura 2: Distintas comisiones para el modelo de 10 observaciones

Teniendo ahora un desempeño mejor en los resultados del modelo.

MODELO CON 3 OBSERVACIONES

Para modificar el modelo y que acepte 3 observaciones se modificó la variable `BARS_COUNT = 3` del archivo train_model.py, de igual manera se modificó el archivo environ.py cambiando la variable `DEFAULT_BARS_COUNT = 3`, se ejecutó con normalidad pero ahora con el comando:

```
python train_model.py -r simple-modelo_3bar
```

El código se dejó corriendo por 12 horas, y para probarlo con los datos de entrenamiento se utilizó el comando:

```
python run_model.py -d data/YNDX_160101_161231.csv -m
saves/simple-modelo_3bar/mean_value-0.124.data -b 3 -n YNDX16
```

Para probarlo con los valores de validación se utilizó:

```
python run_model.py -d data/YNDX_150101_151231.csv -m
saves/simple-modelo_3bar/mean_value-0.124.data -b 3 -n YNDX15
```

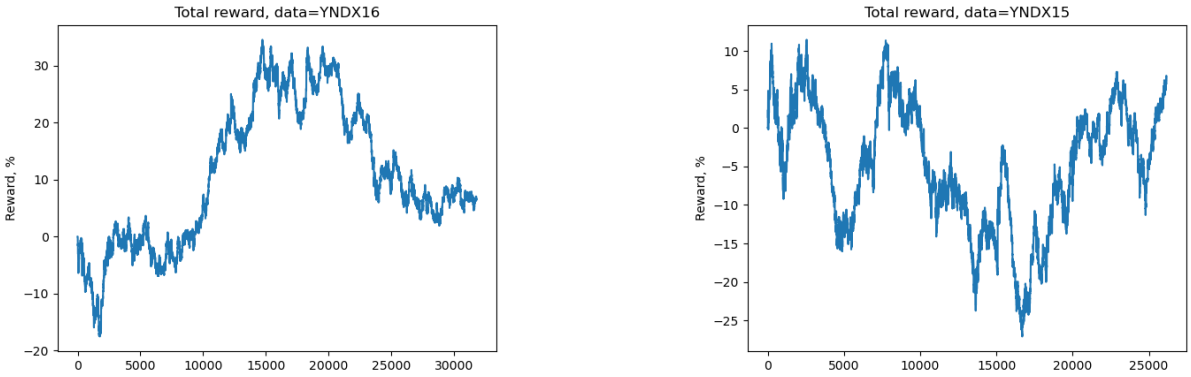


Figura 3: Beneficio comercial del modelo con 3 observaciones en los datos de entrenamiento (izquierda) y datos de validación (derecha)

De manera similar, los datos mejoran al reducir la comisión.

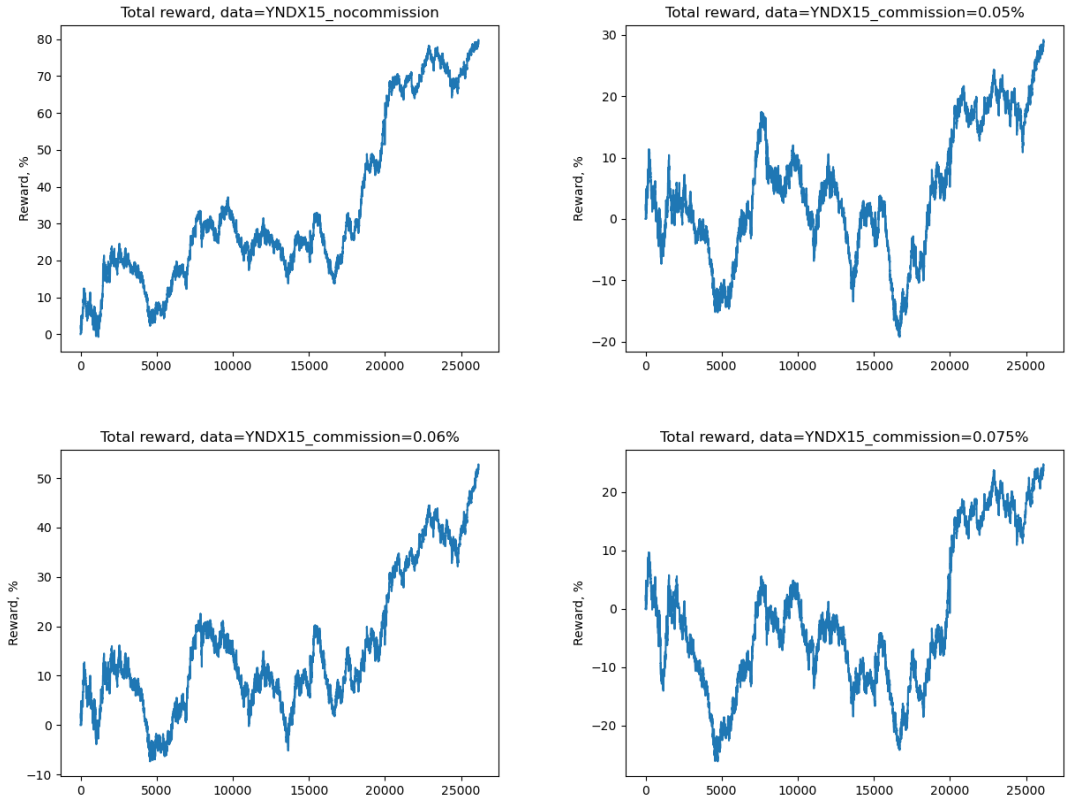


Figura 4: Distintas comisiones para el modelo de 3 observaciones